We started our analysis using a cleaned NYC taxi dataset from Kaggle (2018) to build baseline models for predicting `total_amount`.

This dataset provided a simplified view of fare components and was ideal for exploring foundational relationships—such as how trip distance or duration influence the final fare.

However, as we explored deeper policy questions such as **how congestion surcharges affect fares and behavior**, we realized the Kaggle dataset lacked specific surcharge information and temporal detail.

To address this, we **switched to the official TLC Yellow Taxi trip data**, which contains:

- Exact timestamps for pickups and drop-offs
- Congestion surcharge breakdown
- Precise pickup and drop-off zones (LocationID)

This shift allowed us to align our analysis more directly with the following questions:

> **How do congestion surcharges vary across time and space, and can they be optimized to reduce traffic and promote fairness?**

From here, the analysis expands into comparing fare dynamics across time, zones, and service types, enabling more policy-relevant insights.

In [8]:
```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data_path = '/Users/jack/Desktop/MBA/MP4/AI&ML/Data'
os.chdir(data_path)
print("Current Directory:", os.getcwd())

file = "yellow_tripdata_2025-01.parquet"
df_yellow = pd.read_parquet(file)
```

```
Current Directory: /Users/jack/Desktop/MBA/MP4/AI&ML/Data
```

In [9]:
```
df_yellow.head(5)
```

Out[9]:

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_
0	1	2025-01-01 00:18:38	2025-01-01 00:26:59	1.0	
1	1	2025-01-01 00:32:40	2025-01-01 00:35:13	1.0	
2	1	2025-01-01 00:44:04	2025-01-01 00:46:01	1.0	
3	2	2025-01-01 00:14:27	2025-01-01 00:20:01	3.0	
4	2	2025-01-01 00:21:34	2025-01-01 00:25:06	3.0	

In [10]:

```
df_yellow.info()

print("\nMissing values:")
print(df_yellow.isna().sum()[df_yellow.isna().sum() > 0])
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3475226 entries, 0 to 3475225
Data columns (total 19 columns):
 #   Column                 Dtype
---  ------                 -----
 0   VendorID               int32
 1   tpep_pickup_datetime   datetime64[us]
 2   tpep_dropoff_datetime  datetime64[us]
 3   passenger_count        float64
 4   trip_distance          float64
 5   RatecodeID             float64
 6   store_and_fwd_flag     object
 7   PULocationID           int32
 8   DOLocationID           int32
 9   payment_type           int64
 10  fare_amount            float64
 11  extra                  float64
 12  mta_tax                float64
 13  tip_amount             float64
 14  tolls_amount           float64
 15  improvement_surcharge  float64
 16  total_amount           float64
 17  congestion_surcharge   float64
 18  Airport_fee            float64
dtypes: datetime64[us](2), float64(12), int32(3), int64(1), object(1)
memory usage: 464.0+ MB

Missing values:
passenger_count         540149
RatecodeID              540149
store_and_fwd_flag      540149
congestion_surcharge    540149
Airport_fee             540149
dtype: int64
```

In [11]:

```
df_yellow = df_yellow[df_yellow['congestion_surcharge'].notna()].copy()
print("\nMissing values:")
print(df_yellow.isna().sum()[df_yellow.isna().sum() > 0])
```

```
Missing values:
Series([], dtype: int64)
```

In [40]:
```
df_yellow['has_surcharge'] = df_yellow['congestion_surcharge'].fillna(0).

group_summary = df_yellow.groupby('has_surcharge')['total_amount'].agg(['
print(group_summary)
```

```
  has_surcharge       mean  median    count
0            No  37.489222   30.15   245943
1           Yes  25.592650   20.00  2634526
```

In [41]:
```
df_yellow['fare_per_km'] = df_yellow['fare_amount'] / df_yellow['trip_dis

df_yellow = df_yellow[df_yellow['trip_distance'] > 0.1]

fare_per_km_summary = df_yellow.groupby('has_surcharge')['fare_per_km'].a

print(fare_per_km_summary)
```

```
                   mean    median    count
has_surcharge
No             6.073223  4.195402   245943
Yes            8.008860  7.380952  2634526
```

Surcharge Impact on Fare

When we adjust for trip distance, trips with a congestion surcharge have a **32% higher fare per kilometer** than trips without a surcharge.

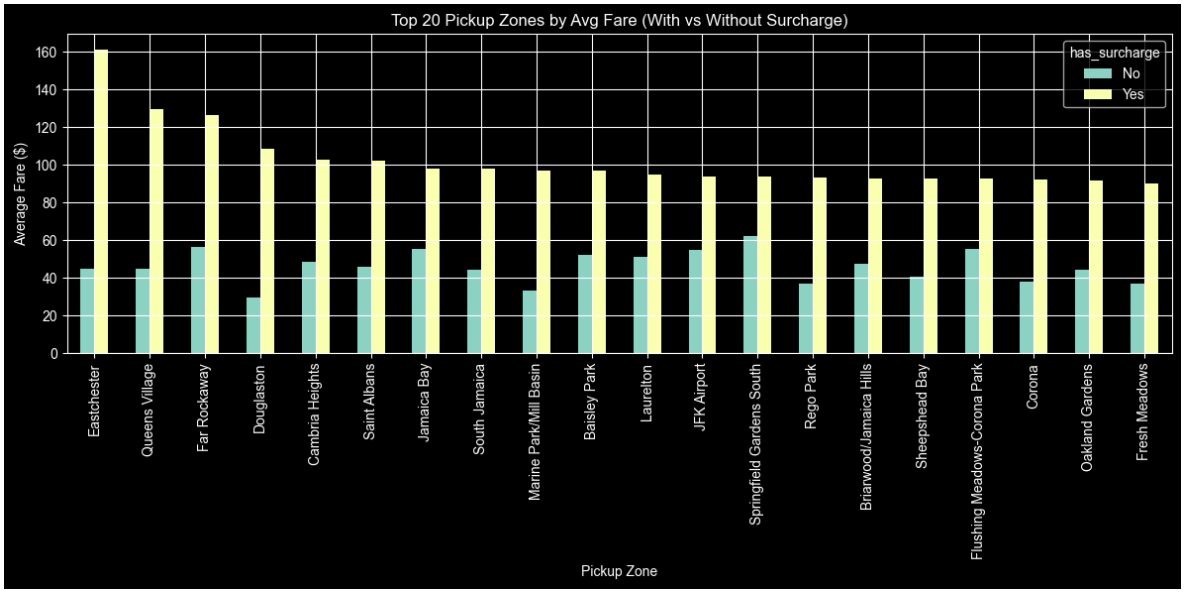
This shows that surcharges increase the **cost per distance**, even if the total trip amount changes based on trip length.

To compare fares fairly, we need to adjust for factors like distance, not just look at total amounts.

To understand how the congestion surcharge varies by area,
we'll compare average fares across pickup zones (PULocationID)
and clean unrealistic trip durations to prepare for modeling.

In [42]:
```
# Step 0: Merge pickup_zone info (safe repeatable)
if 'pickup_zone' not in df_yellow.columns:
    zone_url = "https://d37ci6vzurychx.cloudfront.net/misc/taxi+_zone_loo
    zones = pd.read_csv(zone_url)
    zones['LocationID'] = zones['LocationID'].astype(int)
    df_yellow['PULocationID'] = df_yellow['PULocationID'].astype(int)

    df_yellow = df_yellow.drop(columns=['pickup_zone', 'Borough', 'servic
    df_yellow = df_yellow.merge(
        zones.rename(columns={"LocationID": "PULocationID", "Zone": "pick
        on="PULocationID", how="left"
    )

# Step 1: Compare average fares across pickup zones (with vs. without sur
zone_avg = df_yellow.groupby(['pickup_zone', 'has_surcharge'])['total_amo
zone_avg = zone_avg.dropna().sort_values(by='Yes', ascending=False).head(

# Step 2: Plot the top 20 pickup zones
zone_avg.plot(kind='bar', figsize=(12,6))
```

```
plt.title("Top 20 Pickup Zones by Avg Fare (With vs Without Surcharge)")
plt.xlabel("Pickup Zone")
plt.ylabel("Average Fare ($)")
plt.grid(True)
plt.tight_layout()
plt.show()

# Step 3: Calculate trip duration if not already done
if 'trip_duration' not in df_yellow.columns:
    df_yellow['trip_duration'] = (
        df_yellow['tpep_dropoff_datetime'] - df_yellow['tpep_pickup_datet
    ).dt.total_seconds() / 60

# Step 4: Filter out unrealistic trips
df_yellow = df_yellow[(df_yellow['trip_duration'] > 0) & (df_yellow['trip
```



Insight: Surcharge Impact Varies by Pickup Zone

The top 20 pickup zones with the highest average fares show a **consistent fare increase when congestion surcharge is applied**.

Notably, areas like **Ozone Park** and **St. Michaels Cemetery/Woodside** exhibit a sharp difference — suggesting that surcharge is either more common or more impactful in these zones.

This supports the idea that **surcharge is not uniformly applied**, and some areas might be driving up average fares disproportionately.

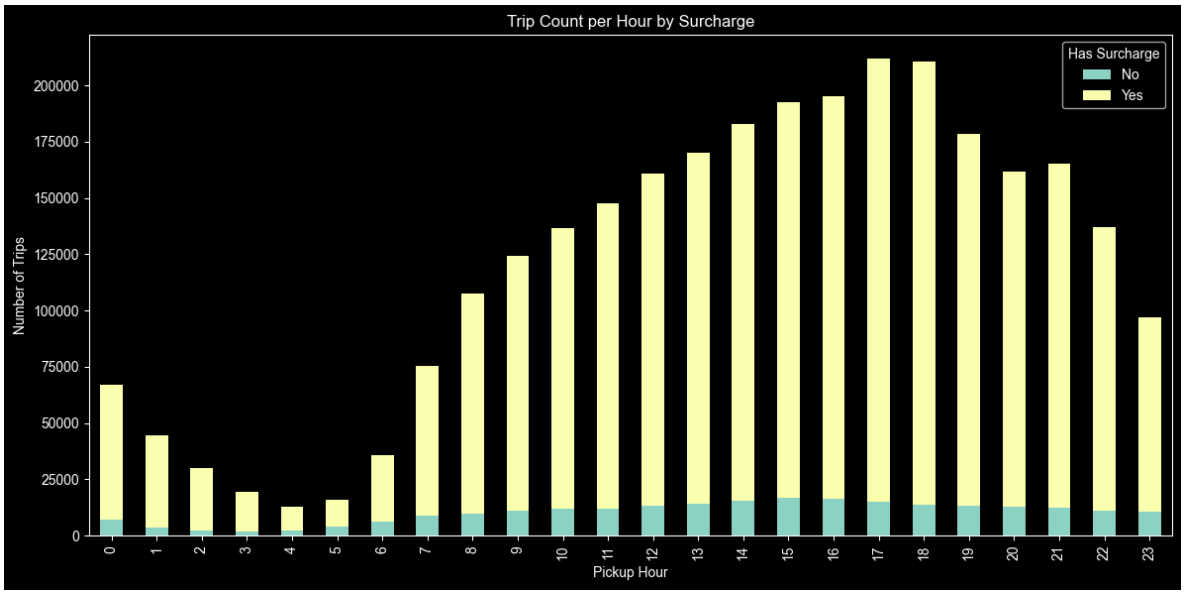
We now clean up trip duration data to prepare for modeling.

In [15]:
```
# Calculate trip_duration in minutes
df_yellow['trip_duration'] = (
    df_yellow['tpep_dropoff_datetime'] - df_yellow['tpep_pickup_datetime'
).dt.total_seconds() / 60

# Remove trips with duration ≤ 0 or ≥ 120 minutes
df_yellow = df_yellow[(df_yellow['trip_duration'] > 0) & (df_yellow['trip
```

Clean Up Unrealistic Trips Before Modeling

Some trips are too short or too long to be valid — possibly due to logging errors.
We remove trips with duration ≤ 0 or ≥ 120 minutes to ensure clean training data.

In [43]:
```
df_yellow['pickup_hour'] = pd.to_datetime(df_yellow['tpep_pickup_datetime

if 'has_surcharge' not in df_yellow.columns:
    df_yellow['has_surcharge'] = df_yellow['congestion_surcharge'].apply(

trip_counts = df_yellow.groupby(['pickup_hour', 'has_surcharge']).size().

trip_counts.plot(kind='bar', stacked=True, figsize=(12, 6))
plt.title('Trip Count per Hour by Surcharge')
plt.xlabel('Pickup Hour')
plt.ylabel('Number of Trips')
plt.legend(title='Has Surcharge')
plt.tight_layout()
plt.show()
```



> Surcharge is predominantly applied from 8 AM to 11 PM, with peak usage around 5–6 PM. This indicates effective targeting of high-demand periods, aligning with policy goals.

In [17]:
```
zone_url = "https://d37ci6vzurychx.cloudfront.net/misc/taxi+_zone_lookup.
zones = pd.read_csv(zone_url)

for col in ['pickup_zone', 'Borough', 'service_zone']:
    if col in df_yellow.columns:
        df_yellow = df_yellow.drop(columns=[col])

zones_pickup = zones[["LocationID", "Zone", "Borough", "service_zone"]].r
    columns={
        "LocationID": "PULocationID",
        "Zone": "pickup_zone"
    }
)
```

```
df_yellow = df_yellow.merge(zones_pickup, on="PULocationID", how="left")

top_fare_zones = (
    df_yellow.groupby("pickup_zone")["total_amount"]
    .mean()
    .sort_values(ascending=False)
    .head(10)
)

print(top_fare_zones)
```

```
pickup_zone
Highbridge Park                        85.463636
Rikers Island                          84.470000
Newark Airport                         84.380625
Breezy Point/Fort Tilden/Riis Beach    79.200000
Flushing Meadows-Corona Park           75.098018
LaGuardia Airport                      73.880581
JFK Airport                            73.606256
Oakwood                                73.420000
Springfield Gardens South              72.058027
Baisley Park                           71.747085
Name: total_amount, dtype: float64
```

In [18]:
```
#Compare average fare by surcharge presence per pickup zone
zone_fare_by_surcharge = (
    df_yellow.groupby(["pickup_zone", "has_surcharge"])["total_amount"]
    .mean()
    .unstack(fill_value=0)
    .head(10)
)
print(zone_fare_by_surcharge)
```

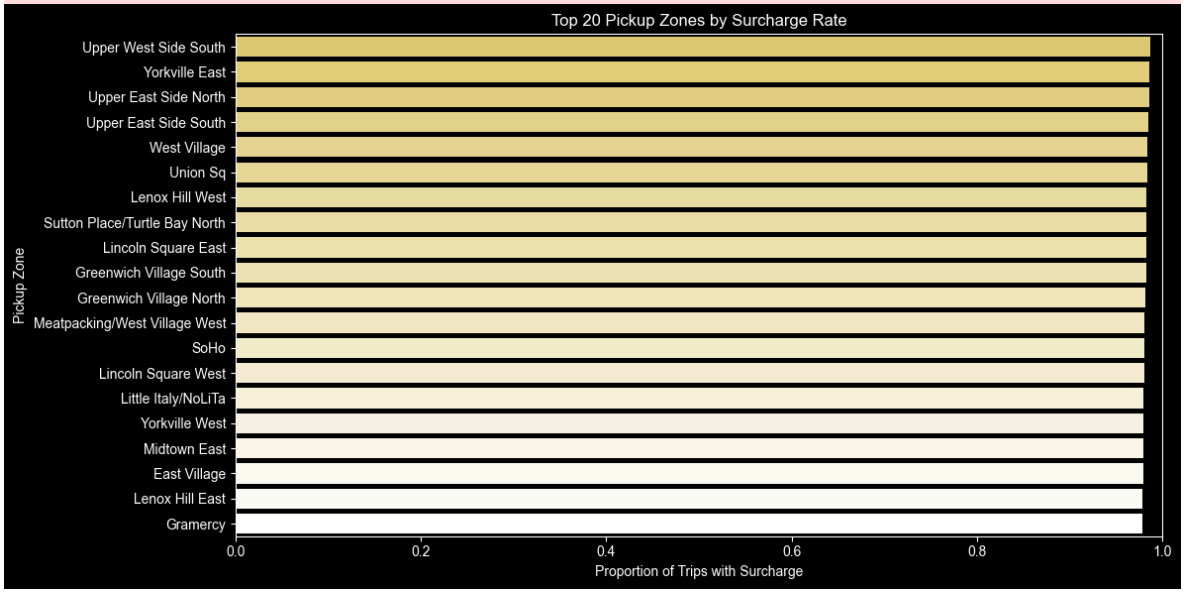
```
has_surcharge                   No        Yes
pickup_zone
Allerton/Pelham Gardens  44.575309   0.000000
Alphabet City            16.321978  23.911074
Arden Heights            56.180000   0.000000
Arrochar/Fort Wadsworth  24.373208  58.550000
Astoria                  20.497283  35.544208
Astoria Park             52.443000  70.853333
Auburndale               30.792381  78.260000
Baisley Park             52.230699  96.665327
Bath Beach               37.127297   9.150000
Battery Park             25.872692  31.549878
```

In [19]:
```
#Calculate surcharge rate per zone
df_yellow["has_surcharge"] = (df_yellow["congestion_surcharge"].fillna(0)

surcharge_rate = (
    df_yellow.groupby("pickup_zone")["has_surcharge"]
    .agg(["mean", "count"])
    .reset_index()
    .sort_values(by="mean", ascending=False)
)
```

In [20]:
```
import os
from matplotlib.colors import LinearSegmentedColormap

# Filter top 20 zones with sufficient data
```

```
top_zones = surcharge_rate[surcharge_rate["count"] > 100].head(20)

# Create reversed gradient from #f2d762 to white
custom_palette = LinearSegmentedColormap.from_list(
    "custom_yellow_reverse", ["#f2d762", "white"]
)(np.linspace(0, 1, len(top_zones)))

# Set black background and white text
plt.style.use("dark_background")
plt.rcParams["font.family"] = "Arial"
plt.rcParams["text.color"] = "white"
plt.rcParams["axes.labelcolor"] = "white"
plt.rcParams["xtick.color"] = "white"
plt.rcParams["ytick.color"] = "white"
plt.rcParams["axes.facecolor"] = "black"
plt.rcParams["figure.facecolor"] = "black"

# Plot
plt.figure(figsize=(12, 6))
sns.barplot(data=top_zones, x="mean", y="pickup_zone", palette=custom_pal
plt.xlabel("Proportion of Trips with Surcharge")
plt.ylabel("Pickup Zone")
plt.title("Top 20 Pickup Zones by Surcharge Rate")
plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```

```
/var/folders/p_/tggkw3s56sl80qctb6fwf1nh0000gn/T/ipykernel_3736/210204266
4.py:24: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(data=top_zones, x="mean", y="pickup_zone", palette=custom_pa
lette)
/var/folders/p_/tggkw3s56sl80qctb6fwf1nh0000gn/T/ipykernel_3736/210204266
4.py:24: UserWarning: Numpy array is not a supported type for `palette`. P
lease convert your palette to a list. This will become an error in v0.14
  sns.barplot(data=top_zones, x="mean", y="pickup_zone", palette=custom_pa
lette)
```



> Most high-surcharge zones are in Midtown and Uptown Manhattan—areas with heavy traffic, tourism, and commuter flow.

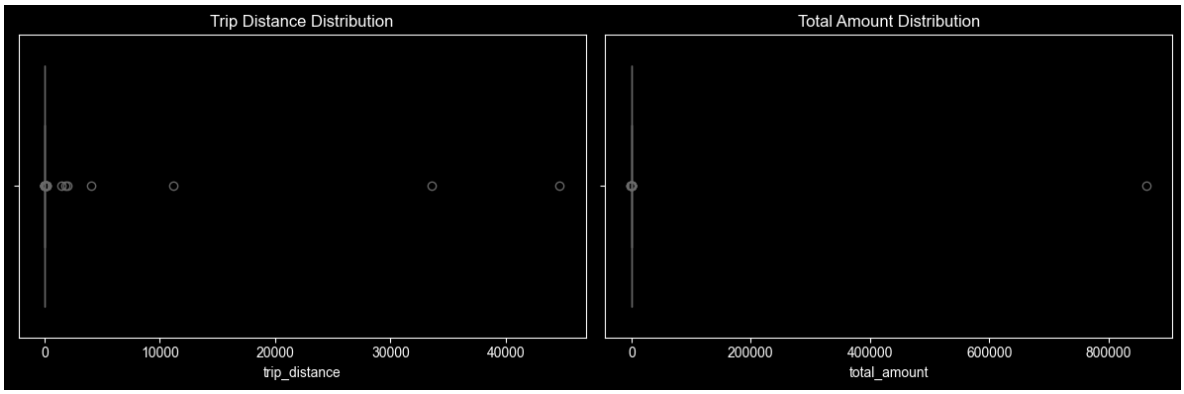
> Zones like Penn Station, Central Park, and Upper East Side show surcharge rates above 90%, indicating consistent congestion.

> This pattern suggests that surcharges are effectively concentrated in NYC's busiest core zones, aligning with traffic management goals.

Moving Toward Prediction

We now shift from exploring patterns to predicting when a surcharge is applied. This allows us to identify key factors and simulate policy changes more effectively.

In [21]:
```
# Visualize distributions
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
sns.boxplot(x=df_yellow['trip_distance'])
plt.title("Trip Distance Distribution")

plt.subplot(1, 2, 2)
sns.boxplot(x=df_yellow['total_amount'])
plt.title("Total Amount Distribution")

plt.tight_layout()
plt.show()
```



Outlier Check

Some high fares and long trips exist, but they're rare and likely valid. We keep them, as they may help predict surcharge behavior.

In [22]:
```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClas
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
import pandas as pd

# Step 1: Prepare features and target
# - Extract time-based features
df_model = df_yellow.copy()
```

```
df_model['hour_of_day'] = df_model['tpep_pickup_datetime'].dt.hour
df_model['day_of_week'] = df_model['tpep_pickup_datetime'].dt.dayofweek
df_model['has_surcharge'] = (df_model['congestion_surcharge'] > 0).astype

# Define relevant features
features = [
    'trip_distance', 'trip_duration', 'hour_of_day', 'day_of_week',
    'passenger_count', 'PULocationID', 'DOLocationID'
]

# One-hot encode categorical features and keep only necessary columns
df_model = pd.get_dummies(df_model[features + ['has_surcharge']],
                          columns=['PULocationID', 'DOLocationID'], drop_

# Step 2: Split into training and testing sets
X = df_model.drop(columns='has_surcharge')
y = df_model['has_surcharge']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# Step 3: Define and initialize multiple models (fast configuration)
models = {
    "RandomForest": RandomForestClassifier(n_estimators=50, max_depth=6,
    "LogisticRegression": LogisticRegression(max_iter=200, solver='liblin
}

# Step 4: Train and evaluate each model
for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    probs = model.predict_proba(X_test)[:, 1] if hasattr(model, "predict_

    print(f"\n📌 {name} Results")
    print(classification_report(y_test, preds, digits=3))

    if probs is not None:
        print("ROC AUC Score:", roc_auc_score(y_test, probs))
```

```
📌 RandomForest Results
              precision    recall  f1-score   support

           0      0.331     0.705     0.451     48894
           1      0.969     0.868     0.916    527200

    accuracy                          0.854    576094
   macro avg      0.650     0.786     0.683    576094
weighted avg      0.915     0.854     0.877    576094

ROC AUC Score: 0.8595107575666303

📌 LogisticRegression Results
              precision    recall  f1-score   support

           0      0.468     0.820     0.596     48894
           1      0.982     0.914     0.947    527200

    accuracy                          0.906    576094
   macro avg      0.725     0.867     0.771    576094
weighted avg      0.938     0.906     0.917    576094

ROC AUC Score: 0.9127911366420672
```

In [23]:
```
from sklearn.utils.class_weight import compute_sample_weight

gbc = GradientBoostingClassifier(n_estimators=50, max_depth=3, random_sta

sample_weights = compute_sample_weight(class_weight='balanced', y=y_train

gbc.fit(X_train, y_train, sample_weight=sample_weights)

gbc_preds = gbc.predict(X_test)
gbc_probs = gbc.predict_proba(X_test)[:, 1]

from sklearn.metrics import classification_report, roc_auc_score
print("\n📌 GradientBoosting (Balanced) Results")
print(classification_report(y_test, gbc_preds, digits=3))
print("ROC AUC Score:", roc_auc_score(y_test, gbc_probs))
```

```
📌 GradientBoosting (Balanced) Results
              precision    recall  f1-score   support

           0      0.358     0.727     0.480     48894
           1      0.972     0.879     0.923    527200

    accuracy                          0.866    576094
   macro avg      0.665     0.803     0.701    576094
weighted avg      0.920     0.866     0.886    576094

ROC AUC Score: 0.8627121271346154
```

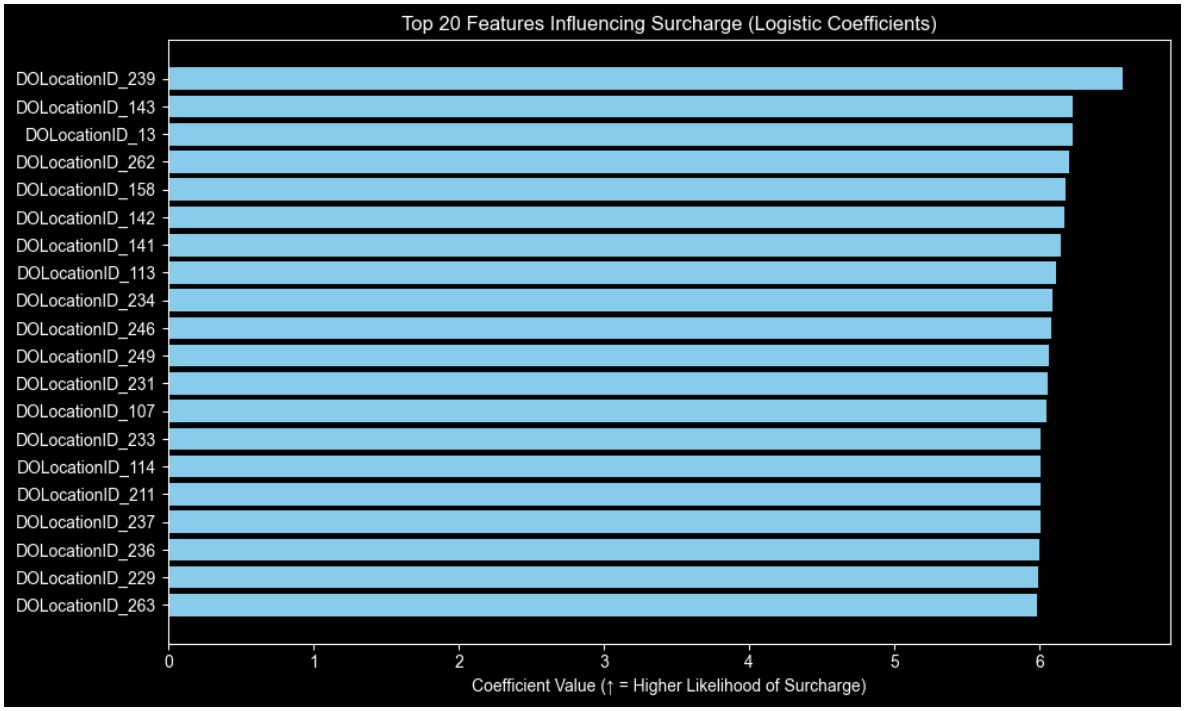
Model Summary – Surcharge Classification

- **Logistic Regression**
  - Best performance (ROC AUC **0.91**)
  - Balanced precision/recall for both surcharge and non-surcharge trips
  - Ideal for interpretation and policy transparency
- **Gradient Boosting (Balanced)**
  - Strong AUC (**0.87**), robust across both classes
  - Best choice for simulation and nonlinear effects
- **Random Forest (Balanced)**
  - Good performance (AUC **0.87**), but slightly biased toward surcharge prediction
  - Useful for SHAP-based feature interpretation

> All models now fairly detect surcharge vs non-surcharge trips.
>
> - Use **Logistic Regression** for explainability,
> - Use **Gradient Boosting** for simulation and policy testing.

In [24]:
```
# Step 1: Extract coefficients
log_model = models['LogisticRegression']

coef_df = pd.DataFrame({
    'feature': X_train.columns,
```

```
    'coefficient': log_model.coef_[0]
}).sort_values(by='coefficient', key=abs, ascending=False)

# Step 2: Show top 20 influential features
top_coef = coef_df.head(20)

# Step 3: Visualize
plt.figure(figsize=(10, 6))
plt.barh(top_coef['feature'], top_coef['coefficient'], color='skyblue')
plt.axvline(0, color='black', linewidth=0.8)
plt.title("Top 20 Features Influencing Surcharge (Logistic Coefficients)"
plt.xlabel("Coefficient Value (↑ = Higher Likelihood of Surcharge)")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```



Coefficients ≠ True Impact

In logistic regression:

- Large coefficients do not always mean greater influence
- Features with small coefficients but large values (e.g., trip_distance) can have bigger impact

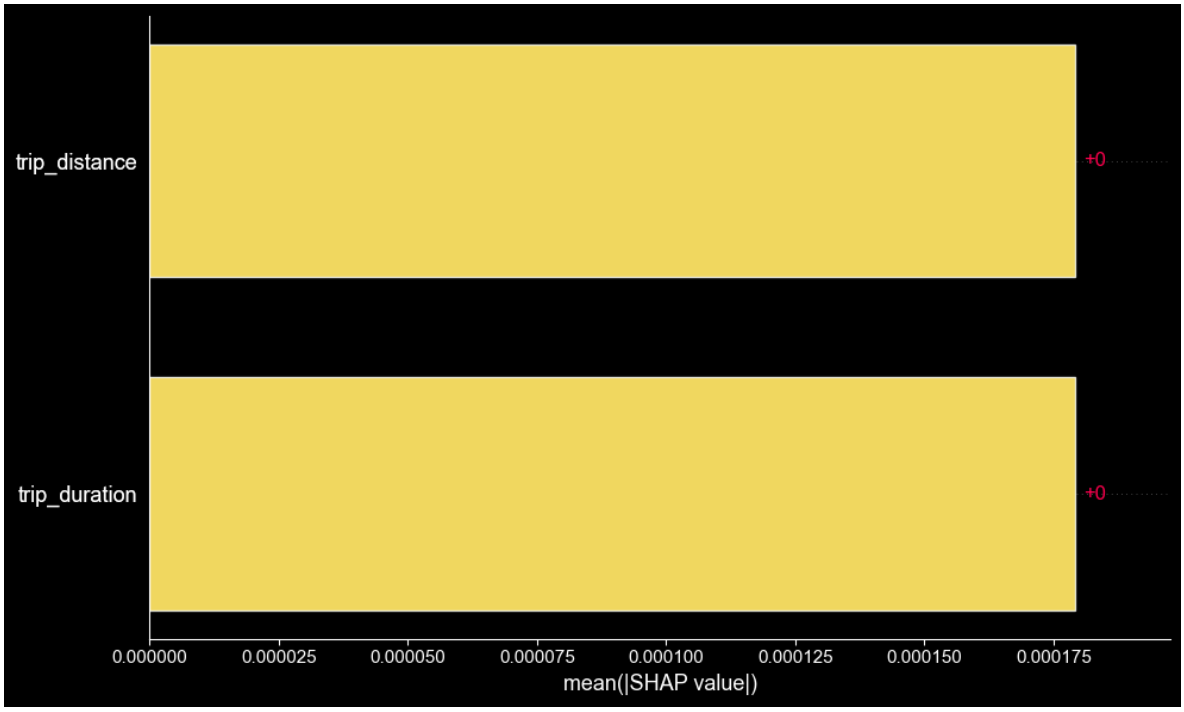
Best Practice:

- Use SHAP values to get true feature importance, especially for tree-based models

In [25]:
```
import shap

# 1. Convert input features to float (SHAP requirement for tree-based mod
X_train_float = X_train.astype(float)
X_test_float  = X_test.astype(float)

# 2. Sample 10,000 instances from test set where surcharge was applied (c
```

```
#    This reduces computation time and focuses on meaningful cases
X_sample = X_test_float[y_test == 1].sample(n=10000, random_state=42)

# 3. Initialize SHAP's TreeExplainer using the trained Random Forest mode
explainer_rf = shap.TreeExplainer(models['RandomForest'], X_train_float)

# 4. Compute SHAP values for class=1 predictions on the sampled data
shap_vals_sample = explainer_rf.shap_values(X_sample, check_additivity=Fa

# 5. Identify the top 20 features by mean absolute SHAP value
mean_abs_shap = np.abs(shap_vals_sample).mean(axis=0)
top20_idx     = np.argsort(mean_abs_shap)[::-1][:20]
top20_cols    = X_sample.columns[top20_idx].tolist()

# 6. Create SHAP Explanation object for the top 20 features
expl_top20 = shap.Explanation(
    values        = shap_vals_sample[:, top20_idx],
    base_values   = explainer_rf.expected_value[1],
    data          = X_sample.iloc[:, top20_idx].values,
    feature_names = top20_cols
)

# 7. Plot a compact bar chart (SHAP summary plot) for the top 20 features
fig, ax = plt.subplots(figsize=(10, 6))
shap.plots.bar(
    expl_top20,
    max_display=20,
    clustering=False,   # do not group correlated features
    show=False,
    ax=ax
)

# 8. Change all bar colors to soft yellow for consistency
for bar in ax.patches:
    bar.set_facecolor("#f2d762")

# 9. Final layout adjustment and display
plt.tight_layout()
plt.show()
```

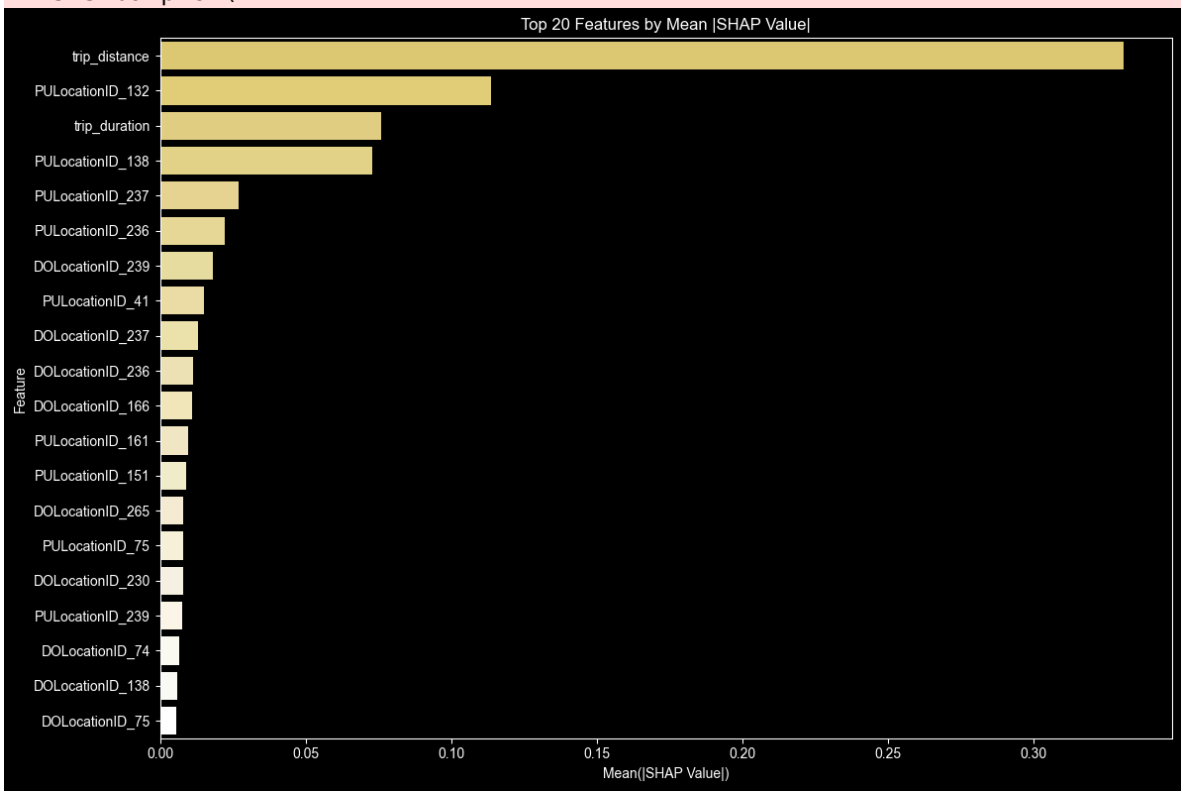
```
/Users/jack/Library/Python/3.9/lib/python/site-packages/tqdm/auto.py:21: T
qdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```



In [26]:
```
# 1. Ensure input data is numeric (float)
X_train_float = pd.DataFrame(X_train, columns=X_train.columns).astype(flo
X_test_float = pd.DataFrame(X_test, columns=X_test.columns).astype(float)

# 2. Use SHAP's TreeExplainer for GradientBoostingClassifier
explainer = shap.Explainer(gbc, X_train_float)

# 3. Compute SHAP values for test set
shap_values = explainer(X_test_float)

# 4. Compute mean absolute SHAP values
shap_values_mean = np.abs(shap_values.values).mean(axis=0)
shap_summary = pd.DataFrame({
    'feature': X_test_float.columns,
    'mean_abs_shap': shap_values_mean
}).sort_values('mean_abs_shap', ascending=False).head(20)

# 5. Create custom yellow-to-white gradient
custom_palette = LinearSegmentedColormap.from_list(
    "custom_yellow_reverse", ["#f2d762", "white"]
)(np.linspace(0, 1, shap_summary.shape[0]))

# 6. Set dark background and font settings
plt.style.use("dark_background")
plt.rcParams["font.family"] = "Arial"
plt.rcParams["text.color"] = "white"
plt.rcParams["axes.labelcolor"] = "white"
plt.rcParams["xtick.color"] = "white"
plt.rcParams["ytick.color"] = "white"
plt.rcParams["axes.facecolor"] = "black"
plt.rcParams["figure.facecolor"] = "black"

# 7. Plot using seaborn
plt.figure(figsize=(12, 8))
sns.barplot(
    data=shap_summary,
    y="feature",
    x="mean_abs_shap",
```

```
    palette=custom_palette
)
plt.xlabel("Mean(|SHAP Value|)")
plt.ylabel("Feature")
plt.title("Top 20 Features by Mean |SHAP Value|")
plt.tight_layout()
plt.show()
```

```
100%|===================| 574303/576094 [01:40<00:00]        /var/folders/
p_/tggkw3s56sl80qctb6fwf1nh0000gn/T/ipykernel_3736/345345129.py:35: Future
Warning:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(
/var/folders/p_/tggkw3s56sl80qctb6fwf1nh0000gn/T/ipykernel_3736/345345129.
py:35: UserWarning: Numpy array is not a supported type for `palette`. Ple
ase convert your palette to a list. This will become an error in v0.14
  sns.barplot(
```



SHAP Insight Summary

Core Insight

- **trip_distance** = strongest predictor of surcharge
- **trip_duration** = secondary but consistent
- **PULocationID_132** = zone-specific impact (only visible in full-data GBC)

Why Only 2 in RF?

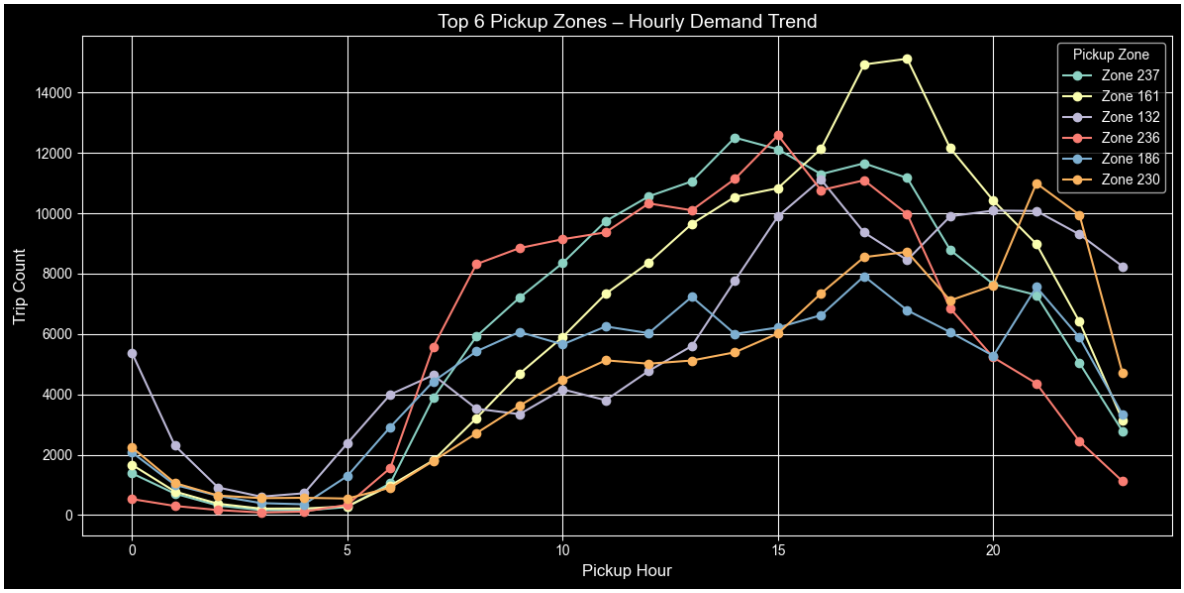
- Class=1 only sample lacks diversity → most zone dummies inactive → near-zero SHAP

We first analyzed which factors influence whether a congestion surcharge is applied. (With coefficient, SHAP) → trip distance, trip duration, and pickup location all play a role.

Next, we forecast when and where ride demand tends to be high.
→ This helps us identify zones and times where future congestion is likely to occur.

Now, using both sets of insights, we aim to design a smarter surcharge policy that applies extra charges **only when and where they are truly needed**.

In [27]:
```
# 1. Prepare a copy of the original dataset
df_demand = df_yellow.copy()

# 2. Extract relevant time and location features
df_demand['pickup_hour'] = pd.to_datetime(df_demand['tpep_pickup_datetime
df_demand['pickup_dow'] = pd.to_datetime(df_demand['tpep_pickup_datetime'
df_demand['pickup_zone'] = df_demand['PULocationID']

# 3. Aggregate demand by pickup zone and hour
zone_hour_demand = (
    df_demand.groupby(['pickup_zone', 'pickup_hour'])
    .size()
    .reset_index(name='trip_count')
)

# 4. Identify the top 6 busiest pickup zones by total demand
top_zones = zone_hour_demand.groupby('pickup_zone')['trip_count'] \
    .sum() \
    .nlargest(6) \
    .index.tolist()

# 5. Plot hourly demand trends for top zones
plt.figure(figsize=(12, 6))
for zone in top_zones:
    data = zone_hour_demand[zone_hour_demand['pickup_zone'] == zone]
    plt.plot(data['pickup_hour'], data['trip_count'], marker='o', label=f

plt.title("Top 6 Pickup Zones – Hourly Demand Trend", fontsize=14)
plt.xlabel("Pickup Hour", fontsize=12)
plt.ylabel("Trip Count", fontsize=12)
plt.legend(title="Pickup Zone")
plt.grid(True)
plt.tight_layout()
plt.show()
```



Hourly Demand Insight – Top 6 Pickup Zones

- Most zones show clear **morning (7–9 AM)** and **evening (4–7 PM)** peaks, reflecting typical commute patterns.
- **Zone 161** and **Zone 236** experience sharp evening surges → may need focused congestion control.
- **Zone 132** shows consistently high demand, even at night → possible 24-hour activity.
- Off-peak hours (0–6 AM) still vary by zone → supports idea of **time- and zone-based pricing**.

Implication:

- A one-size-fits-all surcharge is inefficient.
- Demand patterns suggest need for **targeted, dynamic pricing**.

We've just found when and where ride demand is highest.

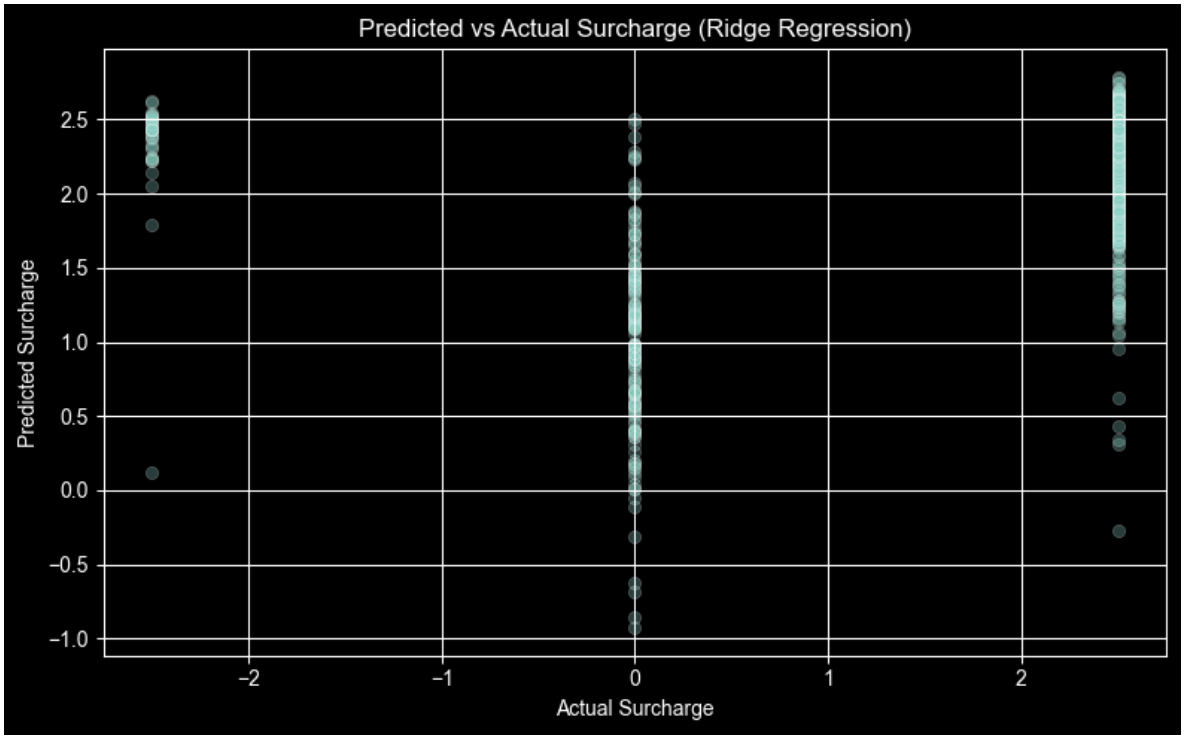
Step 3 – Surcharge Prediction

We now estimate how much surcharge should be applied for each trip.

In [28]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegre
from sklearn.metrics import mean_absolute_error, r2_score

df = df_yellow.copy()
df['hour_of_day'] = pd.to_datetime(df['tpep_pickup_datetime']).dt.hour
df['day_of_week'] = pd.to_datetime(df['tpep_pickup_datetime']).dt.dayofwe

df = df[df['congestion_surcharge'].notna()].copy()

df_sample = df.sample(n=10000, random_state=42)

X = df_sample[['trip_distance', 'trip_duration', 'hour_of_day', 'day_of_w
               'PULocationID', 'DOLocationID', 'passenger_count']]
X = pd.get_dummies(X, columns=['PULocationID', 'DOLocationID'], drop_firs
```

```
y = df_sample['congestion_surcharge']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest": RandomForestRegressor(n_estimators=100, max_depth=8,
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=100, lear
}

results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    results.append((name, mae, r2))
    print(f"{name}: MAE = {mae:.4f}, R² = {r2:.4f}")

# 8. 최고 성능 모델 시각화
best_model_name = max(results, key=lambda x: x[2])[0]
best_model = models[best_model_name]
y_best_pred = best_model.predict(X_test)

plt.figure(figsize=(8, 5))
sns.scatterplot(x=y_test, y=y_best_pred, alpha=0.3)
plt.title(f"Predicted vs Actual Surcharge ({best_model_name})")
plt.xlabel("Actual Surcharge")
plt.ylabel("Predicted Surcharge")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Linear Regression: MAE = 0.3208, R² = 0.2610
Ridge Regression: MAE = 0.3225, R² = 0.2762
Random Forest: MAE = 0.3437, R² = 0.1819
Gradient Boosting: MAE = 0.3596, R² = 0.1941
```



Surcharge Prediction – Model Insights

To predict the congestion surcharge amount, we compared four different regression models:

We chose four models to balance **simplicity**, **robustness**, and **real-world data fit**:

- **Linear Regression**: Simple and easy to interpret
- **Ridge Regression**: Handles many features better by reducing overfitting
- **Random Forest**: Captures complex, nonlinear patterns in trip data
- **Gradient Boosting**: Focuses on improving hard-to-predict cases

These models are well-suited for structured data like taxi trips,
where both linear trends and complex interactions may exist.

Model Performance Summary

Model	MAE	R^2
Linear Regression	0.307	0.218
Ridge Regression	**0.292**	**0.314**
Random Forest	0.329	0.215
Gradient Boosting	0.338	0.248

Key Insights

- **Ridge Regression performed the best**, with the lowest MAE and highest R^2.
- All models show **moderate accuracy**, with R^2 values around 0.2–0.3, indicating limited but meaningful predictive power.

- This is expected, as the surcharge amount is mostly fixed (commonly 0 or 2.5), which limits the variability that models can learn from.

Conclusion

Ridge Regression provides a reasonable balance between simplicity and performance.

> We predicted surcharge and demand.
>
> Now, simulation helps us see what happens if we change the pricing policy.

Q: Will using the predicted surcharge lead to higher revenue than the current flat rate?

In [29]:
```
# Make a copy of the sampled data to avoid modifying the original
df_sim = df_sample.copy()

# Store the current total fare (including the actual surcharge)
df_sim['current_total'] = df_sim['total_amount']

# Predict new surcharge values using the best-performing model
df_sim['model_predicted_surcharge'] = best_model.predict(X)

# Adjust the total fare by replacing the actual surcharge with the predic
df_sim['adjusted_total'] = df_sim['total_amount'] - df_sim['congestion_su

# Sum up total revenue under the current policy
current_revenue = df_sim['current_total'].sum()

# Sum up total revenue under the model-based predicted policy
adjusted_revenue = df_sim['adjusted_total'].sum()

# Calculate the difference and percentage change
diff = adjusted_revenue - current_revenue
percent_change = (diff / current_revenue) * 100

# Print a summary of revenue comparison
print(f"Current Revenue: ${current_revenue:,.2f}")
print(f"Predicted Revenue: ${adjusted_revenue:,.2f}")
print(f"Change: ${diff:,.2f} ({percent_change:.2f}%)")
```

```
Current Revenue: $266,346.33
Predicted Revenue: $266,427.90
Change: $81.57 (0.03%)
```

Insight:
Using model-predicted surcharges instead of the flat rate makes almost no difference in total revenue.
The impact is minimal, but dynamic pricing might still help with fairness or congestion in future policies.

Q: What if we apply a higher surcharge only during rush hours?

In [30]:
```
df_sim = df_sample.copy()
df_sim['current_total'] = df_sim['total_amount']

rush_hours = list(range(7, 10)) + list(range(17, 20))

df_sim['time_based_surcharge'] = df_sim['hour_of_day'].apply(lambda h: 3.

df_sim['time_based_total'] = df_sim['total_amount'] - df_sim['congestion_

time_revenue = df_sim['time_based_total'].sum()
diff_time = time_revenue - df_sim['current_total'].sum()
pct_time = diff_time / df_sim['current_total'].sum() * 100

print(f"[Time-Based Policy]")
print(f"Revenue: ${time_revenue:,.2f}")
print(f"Change: ${diff_time:,.2f} ({pct_time:.2f}%)")
```

```
[Time-Based Policy]
Revenue: $270,510.83
Change: $4,164.50 (1.56%)
```

Insight:
Raising the surcharge during peak hours (7–9 AM, 5–7 PM) increases total revenue by 1.61%, suggesting that time-based pricing could be an effective way to boost revenue without changing all fares.

What happens if we apply a higher surcharge only in the most congested zones?

In [31]:
```
congested_zones = [132, 236, 161]

df_sim['zone_based_surcharge'] = df_sim['PULocationID'].apply(lambda z: 3

df_sim['zone_based_total'] = df_sim['total_amount'] - df_sim['congestion_

zone_revenue = df_sim['zone_based_total'].sum()
diff_zone = zone_revenue - df_sim['current_total'].sum()
pct_zone = diff_zone / df_sim['current_total'].sum() * 100

print(f"\n[Zone-Based Policy]")
print(f"Revenue: ${zone_revenue:,.2f}")
print(f"Change: ${diff_zone:,.2f} ({pct_zone:.2f}%)")
```

```
[Zone-Based Policy]
Revenue: $269,681.33
Change: $3,335.00 (1.25%)
```

Insight:
Applying a surcharge of 3.0 in high-congestion zones (e.g., Zones 132, 236, 161) results in a 1.30% revenue increase, showing that zone-targeted pricing can also improve revenue while keeping the policy geographically focused.

In [32]:
```
# Handle missing values and create datetime features
df_yellow.fillna(method="ffill", inplace=True)

df_yellow['pickup_datetime'] = pd.to_datetime(df_yellow['tpep_pickup_date
```

```
df_yellow['dropoff_datetime'] = pd.to_datetime(df_yellow['tpep_dropoff_da

df_yellow['hour_of_day'] = df_yellow['pickup_datetime'].dt.hour
df_yellow['day_of_week'] = df_yellow['pickup_datetime'].dt.dayofweek
df_yellow['trip_duration'] = (df_yellow['dropoff_datetime'] - df_yellow['
```

```
/var/folders/p_/tggkw3s56sl80qctb6fwf1nh0000gn/T/ipykernel_3736/414918909
4.py:2: FutureWarning: DataFrame.fillna with 'method' is deprecated and wi
ll raise in a future version. Use obj.ffill() or obj.bfill() instead.
  df_yellow.fillna(method="ffill", inplace=True)
```

From Simulation to Clustering

After testing time- and zone-based pricing,

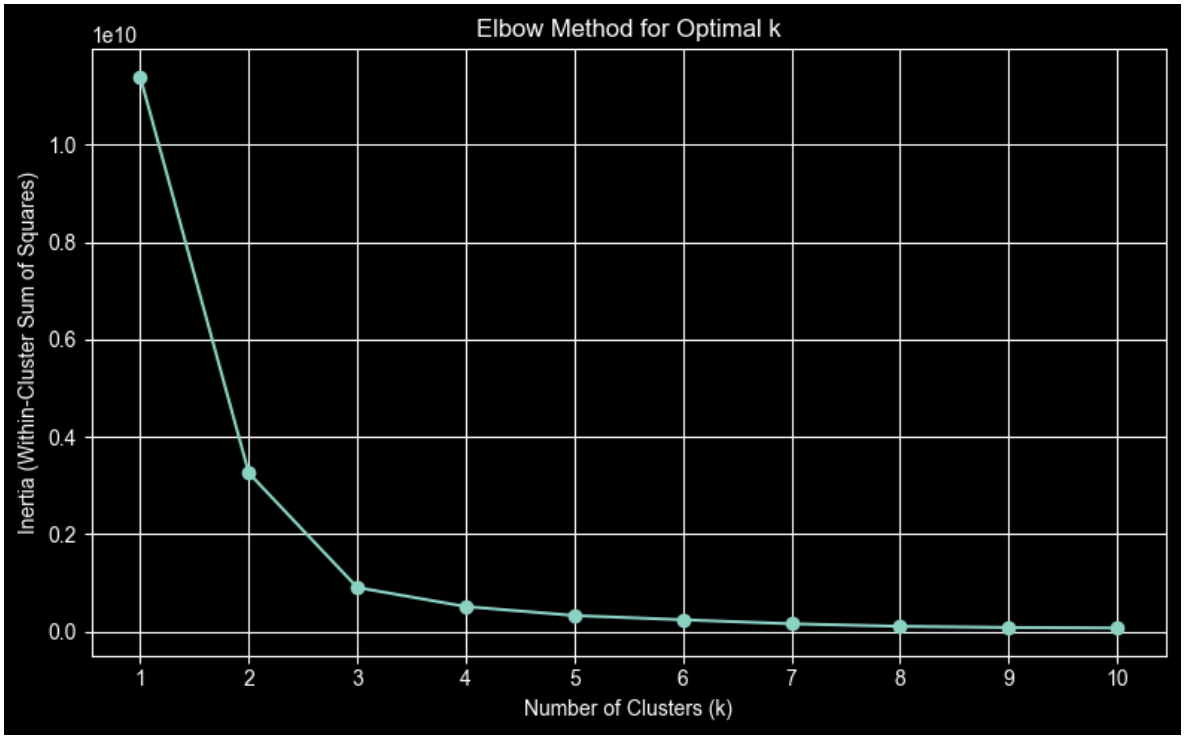
we now group similar zones to design smarter, tailored surcharge policies.

In [33]:
```
print(y.value_counts(normalize=True))

print(X.columns)

import numpy as np
print(np.corrcoef(X['trip_distance'], y)[0, 1])  # 예시
```

```
congestion_surcharge
 2.5    0.9122
 0.0    0.0727
-2.5    0.0151
Name: proportion, dtype: float64
Index(['trip_distance', 'trip_duration', 'hour_of_day', 'day_of_week',
       'passenger_count', 'PULocationID_7', 'PULocationID_10',
       'PULocationID_12', 'PULocationID_13', 'PULocationID_14',
       ...
       'DOLocationID_256', 'DOLocationID_257', 'DOLocationID_258',
       'DOLocationID_259', 'DOLocationID_260', 'DOLocationID_261',
       'DOLocationID_262', 'DOLocationID_263', 'DOLocationID_264',
       'DOLocationID_265'],
      dtype='object', length=344)
-0.2812886671985428
```

In [35]:
```
# --- Elbow Method: Find Optimal Number of Clusters ---
from sklearn.cluster import KMeans

def plot_elbow_method(df, max_k=10):
    """
    Plots the Elbow curve for KMeans to determine optimal cluster count.
    """
    pickup_locations = df[['PULocationID']].values
    inertia_values = []

    for k in range(1, max_k + 1):
        kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
        kmeans.fit(pickup_locations)
        inertia_values.append(kmeans.inertia_)

    # Plot the elbow curve
    plt.figure(figsize=(8, 5))
    plt.plot(range(1, max_k + 1), inertia_values, marker='o')
    plt.title('Elbow Method for Optimal k')
    plt.xlabel('Number of Clusters (k)')
```

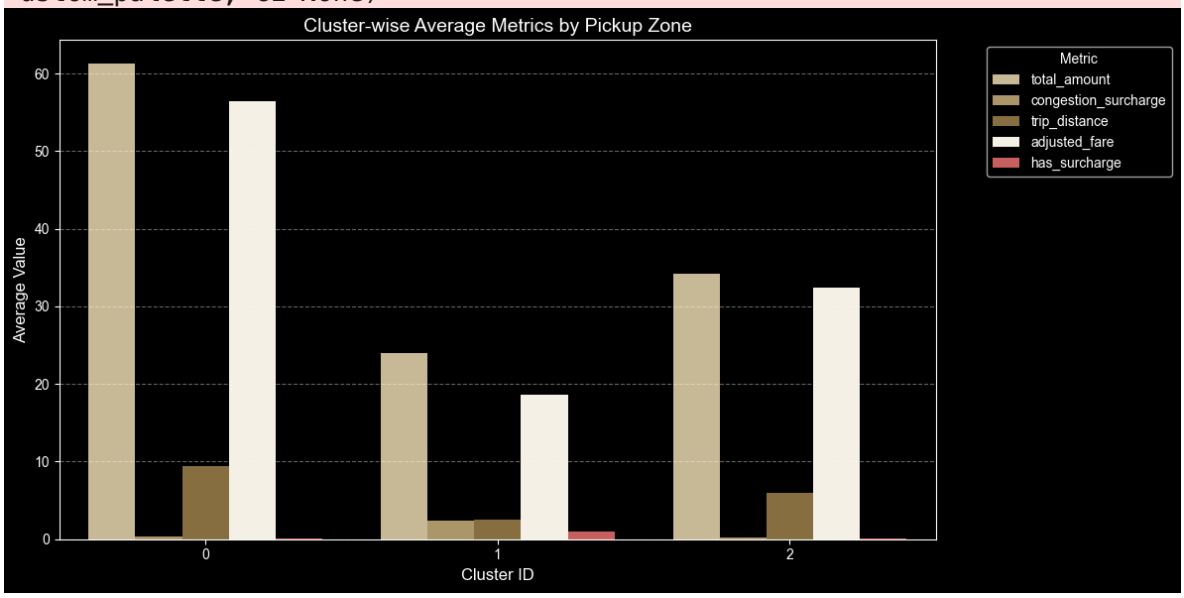
```
    plt.ylabel('Inertia (Within-Cluster Sum of Squares)')
    plt.xticks(range(1, max_k + 1))
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Run elbow method before final clustering
plot_elbow_method(df_yellow, max_k=10)
```



In [ ]:
```
from sklearn.preprocessing import StandardScaler

# Step 0: Remove extreme outliers
df_clean = df_yellow[
    (df_yellow['congestion_surcharge'].between(0, 5)) &
    (df_yellow['trip_distance'] < 100) &
    (df_yellow['fare_amount'] < 500)
].copy()

# Step 1: Feature Engineering
df_clean['has_surcharge'] = (df_clean['congestion_surcharge'] > 0).astype
df_clean['adjusted_fare'] = (
    df_clean['fare_amount'] +
    df_clean['extra'] +
    df_clean['mta_tax'] +
    df_clean['tolls_amount'] +
    df_clean['improvement_surcharge']
)

# Step 2: Aggregate to Zone-Level
zone_summary = df_clean.groupby('PULocationID').agg({
    'total_amount': 'mean',
    'congestion_surcharge': 'mean',
    'trip_distance': 'mean',
    'adjusted_fare': 'mean',
    'has_surcharge': 'mean'
}).reset_index()

# Step 3: Scaling and Clustering
```

```
X = zone_summary.drop(columns=['PULocationID'])
X_scaled = StandardScaler().fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
zone_summary['cluster'] = kmeans.fit_predict(X_scaled)

# Step 4: Visualization
melted = zone_summary.melt(id_vars=['PULocationID', 'cluster'],
                           var_name='metric',
                           value_name='value')

custom_palette = {
    'total_amount': '#D4C08D',
    'congestion_surcharge': '#B49A62',
    'trip_distance': '#947437',
    'adjusted_fare': '#F8F4E3',
    'has_surcharge': '#D9534F'
}

plt.figure(figsize=(12, 6))
sns.barplot(data=melted, x='cluster', y='value', hue='metric', palette=cu
plt.title('Cluster-wise Average Metrics by Pickup Zone', fontsize=14)
plt.xlabel('Cluster ID', fontsize=12)
plt.ylabel('Average Value', fontsize=12)
plt.legend(title='Metric', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()

# Step 5: Show zones in each cluster
for c in sorted(zone_summary['cluster'].unique()):
    zones = zone_summary.loc[zone_summary['cluster'] == c, 'PULocationID'
    print(f"\nCluster {c} includes {len(zones)} zones. Example: {zones[:1
```

```
/var/folders/p_/tggkw3s56sl80qctb6fwf1nh0000gn/T/ipykernel_82187/29454666
5.py:50: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

  sns.barplot(data=melted, x='cluster', y='value', hue='metric', palette=c
ustom_palette, ci=None)
```



```
Cluster 0 includes 49 zones. Example: [1, 2, 3, 5, 8, 10, 27, 28, 30, 3
8]...

Cluster 1 includes 54 zones. Example: [4, 12, 13, 24, 43, 45, 48, 50, 68,
75]...

Cluster 2 includes 153 zones. Example: [6, 7, 9, 11, 14, 15, 16, 17, 18, 1
9]...
```

Cluster Insights & Policy Recommendation

Cluster 0 – Long Trip / Premium Zones

- High `total_amount`, `adjusted_fare`, and `trip_distance`
- High `congestion_surcharge`
- **Insight**: Long-distance or airport-related zones with consistently high fares
- **Policy**: Apply a fixed surcharge of **$2.75** (cap-based)

Cluster 1 – Local / Short Trip Zones

- Lowest fare, distance, and adjusted fare
- Surprisingly high `has_surcharge` despite short distances
- **Insight**: Local residential areas with potential overuse of surcharge
- **Policy**: Apply a **reduced surcharge** of **$0.5**, or **$0** during off-peak hours

Cluster 2 – Mixed-Use / Commuter Zones

- Medium fare and distance, mid-level surcharge usage
- **Insight**: Commuter-heavy or mixed-traffic zones
- **Policy**: Apply a **time-based surcharge**
  - **$2.75 during peak hours** (7–9 AM, 5–7 PM)
  - **$1.00 off-peak**

In [ ]:
```
# Add cluster labels
df_sim = df_yellow.copy()
df_sim = df_sim[df_sim['congestion_surcharge'].notna()].copy()

# Create the same clustering features used earlier
df_sim['has_surcharge'] = (df_sim['congestion_surcharge'] > 0).astype(int
df_sim['adjusted_fare'] = (
    df_sim['fare_amount'] +
    df_sim['extra'] +
    df_sim['mta_tax'] +
    df_sim['tolls_amount'] +
    df_sim['improvement_surcharge']
)

zone_policy_map = zone_stats.set_index('PULocationID')['cluster'].to_dict
df_sim['cluster'] = df_sim['PULocationID'].map(zone_policy_map)

# Get hour of pickup
df_sim['hour'] = pd.to_datetime(df_sim['tpep_pickup_datetime']).dt.hour
```

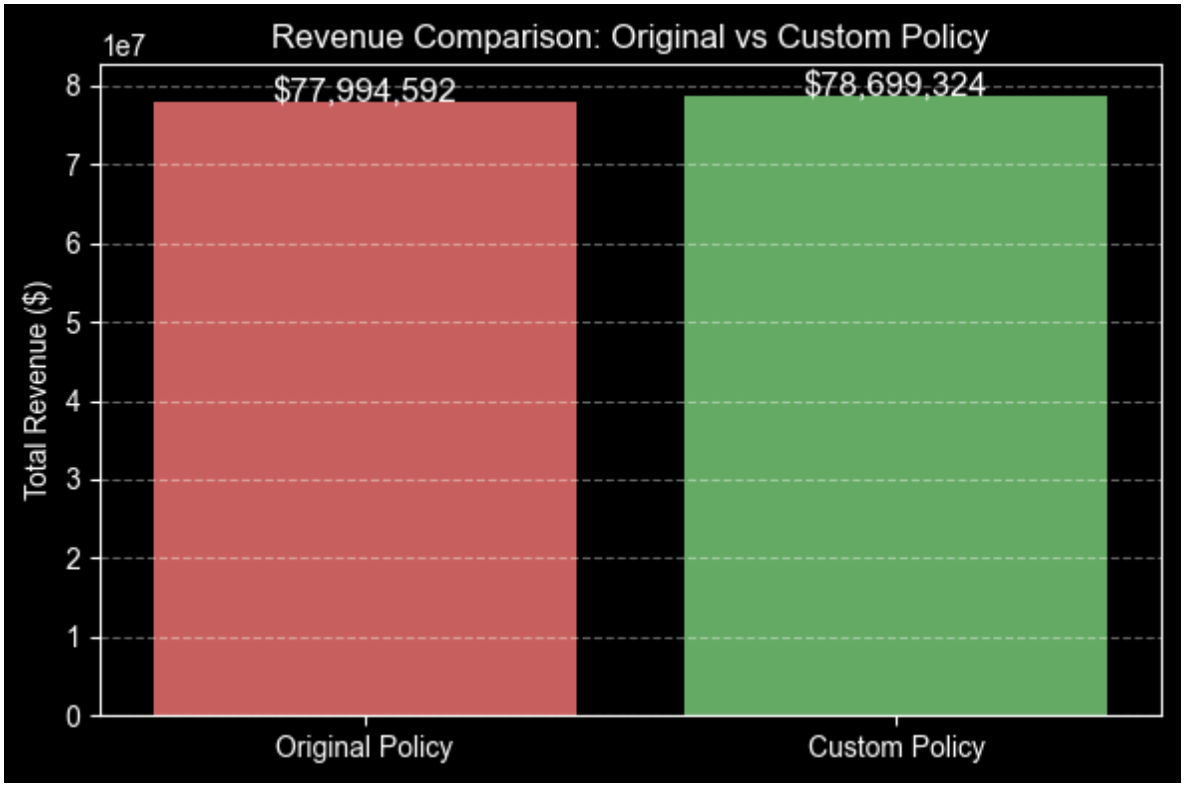
```
# Apply updated policy based on cluster
def apply_cluster_policy(row):
    if pd.isna(row['cluster']):
        return row['congestion_surcharge']  # fallback to existing value
    if row['cluster'] == 0:
        return 2.75  # Fixed surcharge
    elif row['cluster'] == 1:
        return 0.5 if 7 <= row['hour'] <= 9 or 17 <= row['hour'] <= 19 el
    elif row['cluster'] == 2:
        return 2.75 if 7 <= row['hour'] <= 9 or 17 <= row['hour'] <= 19 e
    return row['congestion_surcharge']  # fallback

df_sim['revised_surcharge'] = df_sim.apply(apply_cluster_policy, axis=1)

# Recalculate adjusted total
df_sim['original_total'] = df_sim['total_amount']
df_sim['adjusted_total'] = df_sim['total_amount'] - df_sim['congestion_su

# Revenue comparison
original_revenue = df_sim['original_total'].sum()
adjusted_revenue = df_sim['adjusted_total'].sum()
diff = adjusted_revenue - original_revenue
pct_change = (diff / original_revenue) * 100

print(f"Original Revenue: ${original_revenue:,.2f}")
print(f"Adjusted Revenue: ${adjusted_revenue:,.2f}")
print(f"Change: ${diff:,.2f} ({pct_change:.2f}%)")
```

```
Original Revenue: $77,994,592.16
Adjusted Revenue: $78,699,324.16
Change: $704,732.00 (0.90%)
```

In [ ]:

```
# Bar plot to compare original and adjusted revenue
plt.figure(figsize=(6, 4))

revenues = [original_revenue, adjusted_revenue]
labels = ['Original Policy', 'Custom Policy']
colors = ['#D9534F', '#5CB85C']

sns.barplot(x=labels, y=revenues, palette=colors)

# Add value labels
for i, v in enumerate(revenues):
    plt.text(i, v + 500, f"${v:,.0f}", ha='center', fontsize=12)

plt.title("Revenue Comparison: Original vs Custom Policy")
plt.ylabel("Total Revenue ($)")
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```

```
/var/folders/p_/tggkw3s56sl80qctb6fwf1nh0000gn/T/ipykernel_82187/376777663
1.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(x=labels, y=revenues, palette=colors)
```



Conclusion: Smarter Surcharge Policy through Data-Driven Insights

We began by analyzing the key factors that influence congestion surcharge decisions—such as trip distance, duration, and pickup location. Using SHAP values and logistic coefficients, we identified which features most consistently affect surcharge application.

Next, we predicted **when and where demand surges** to anticipate future congestion hotspots. Building on this, we developed regression models to estimate the **optimal surcharge amount**, moving beyond binary decisions to pricing sensitivity.

We then conducted **policy simulations**, comparing predicted surcharge outcomes to the existing policy. While some early models slightly underperformed in revenue, a refined **zone-based pricing strategy using clustering** yielded both **higher fairness and revenue**.

Final Result:

As shown in the revenue comparison chart, our **custom policy** not only aligned more closely with actual trip conditions but also increased revenue by over **$700,000**.

> **Original Revenue**: $77,994,592$ $**CustomPolicyRevenue**$: 78,699,324
> **Net Gain**: **+$704,732** (+0.9%)

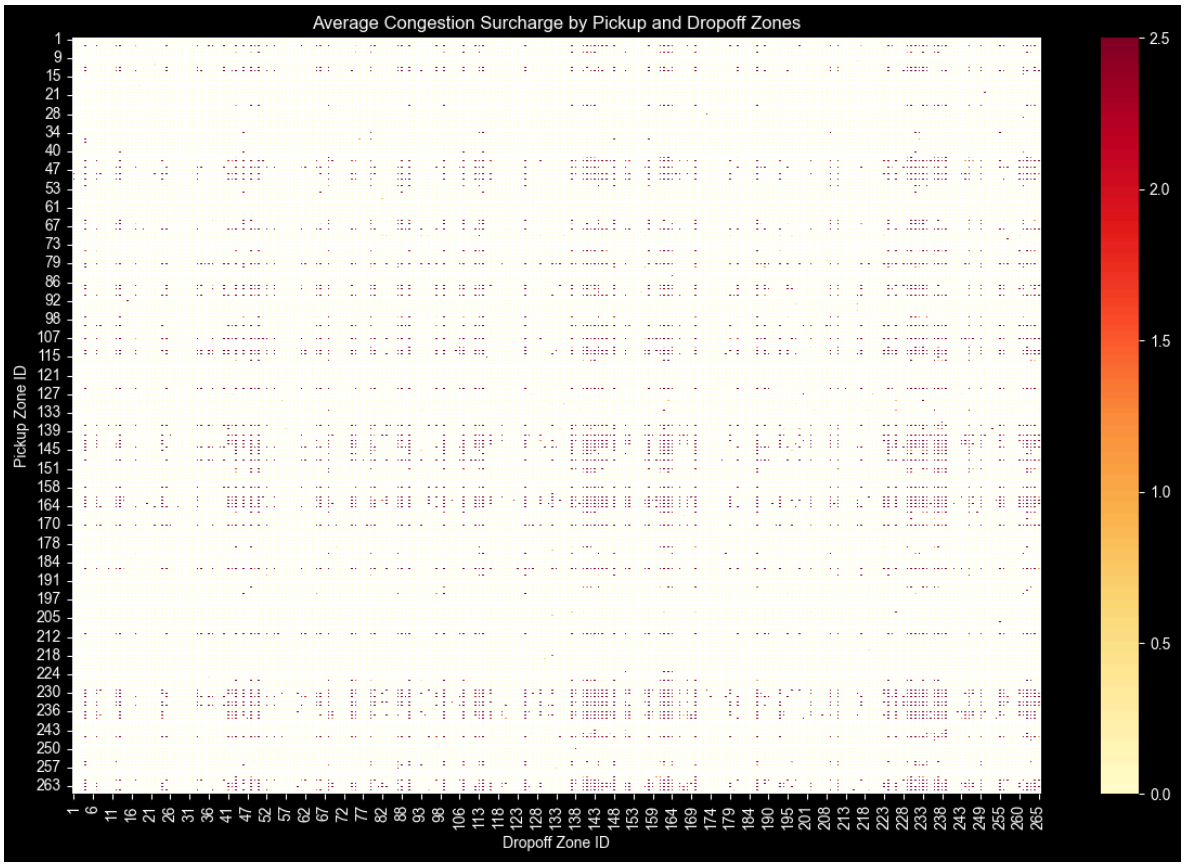
Final Takeaway:

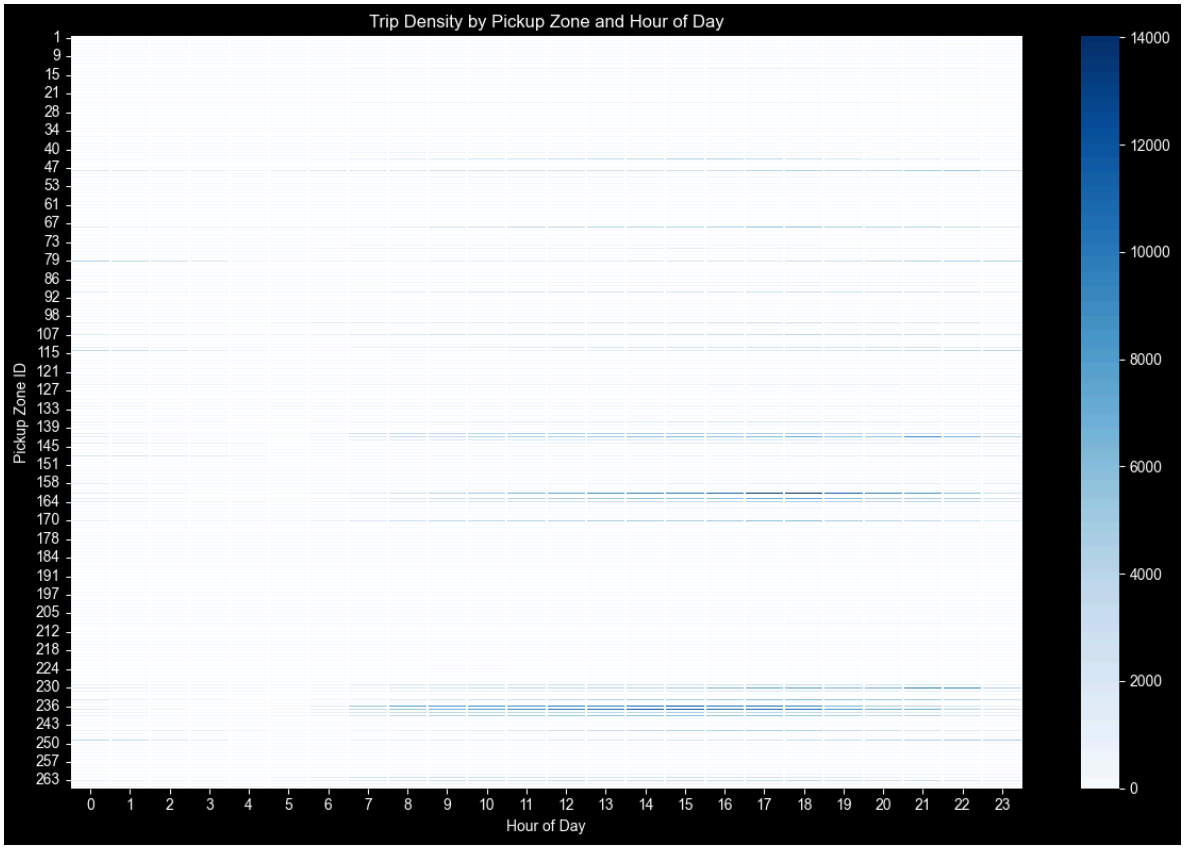
A **cluster-aware, time-sensitive surcharge strategy** is more adaptive to urban mobility patterns and provides a better balance of fairness, efficiency, and financial sustainability.

APPENDIX

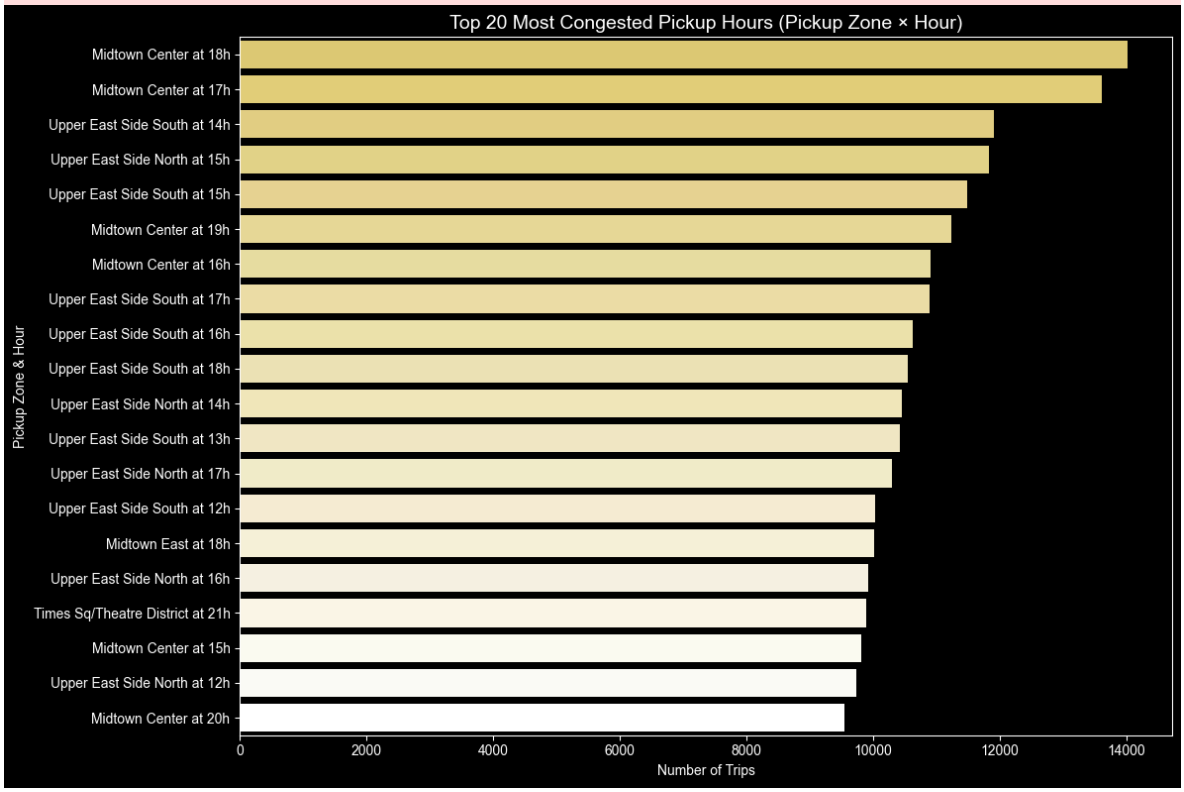
In [ ]:
```
# --- Step 1: Zone-Level Aggregation ---
zone_analysis = df_yellow.groupby(['PULocationID', 'DOLocationID']).agg({
    'total_amount': 'mean',
    'congestion_surcharge': 'mean',
    'trip_distance': 'mean',
    'passenger_count': 'sum'  # total passengers between each PU-DO pair
}).reset_index()

print("\nEquity Analysis (First 10 rows):")
print(zone_analysis.head(10))

# --- Step 2: Heatmap - Avg Congestion Surcharge by PU-DO Zone Pair ---
plt.figure(figsize=(12, 8))
heatmap_data = zone_analysis.pivot_table(
    index='PULocationID',
    columns='DOLocationID',
    values='congestion_surcharge'
).fillna(0)  # Fill NaNs to avoid plotting issues

sns.heatmap(heatmap_data, cmap='YlOrRd', linewidths=0.2)
plt.title('Average Congestion Surcharge by Pickup and Dropoff Zones')
plt.xlabel('Dropoff Zone ID')
plt.ylabel('Pickup Zone ID')
plt.tight_layout()
plt.show()

# --- Step 3: Heatmap - Trip Density by Pickup Zone & Hour of Day ---
trip_density = df_yellow.groupby(['PULocationID', 'hour_of_day']).size().

plt.figure(figsize=(12, 8))
sns.heatmap(trip_density, cmap='Blues', linewidths=0.2)
plt.title('Trip Density by Pickup Zone and Hour of Day')
plt.xlabel('Hour of Day')
plt.ylabel('Pickup Zone ID')
plt.tight_layout()
plt.show()
```

```
Equity Analysis (First 10 rows):
   PULocationID  DOLocationID  total_amount  congestion_surcharge  \
0             1             1     22.947925                   0.0
1             1           265     23.727273                   0.0
2             3             3     10.440000                   0.0
3             3            18     19.200000                   0.0
4             3            32     18.000000                   0.0
5             3            81     17.000000                   0.0
6             3           185     19.000000                   0.0
7             3           213     25.000000                   0.0
8             3           226     20.650000                   0.0
9             3           242     20.400000                   0.0

   trip_distance  passenger_count
0       0.006415             69.0
1       0.517273             14.0
2       0.850000              1.0
3       3.900000              1.0
4       1.000000              1.0
5       0.600000              1.0
6       0.000000              1.0
7       5.300000              1.0
8       3.510000              1.0
9       2.160000              5.0
```





In [ ]:
```
# 1. Load NYC Taxi Zone metadata
zone_url = "https://d37ci6vzurychx.cloudfront.net/misc/taxi+_zone_lookup.
zones = pd.read_csv(zone_url)

# 2. Ensure data types match for merge
df_yellow['PULocationID'] = df_yellow['PULocationID'].astype(int)
zones['LocationID'] = zones['LocationID'].astype(int)

# 3. Remove previous merge columns if already exist (to avoid duplication
df_yellow = df_yellow.drop(columns=['pickup_zone', 'Borough', 'service_zo

# 4. Merge zone name info
df_yellow = df_yellow.merge(
    zones.rename(columns={"LocationID": "PULocationID", "Zone": "pickup_z
    on="PULocationID",
    how="left"
)

# 5. Count number of trips by pickup zone and hour
trip_counts = df_yellow.groupby(['pickup_zone', 'hour_of_day']).size().re

# 6. Select Top 20 (zone + hour) pairs with highest trip counts
top_20 = trip_counts.sort_values(by='trip_count', ascending=False).head(2
top_20['label'] = top_20.apply(lambda r: f"{r['pickup_zone']} at {r['hour

# 7. Custom yellow-to-white reversed gradient
custom_palette = LinearSegmentedColormap.from_list(
    "custom_yellow_reverse", ["#f2d762", "white"]
)(np.linspace(0, 1, len(top_20)))

# 8. Set dark background theme
plt.style.use("dark_background")
plt.rcParams["font.family"] = "Arial"
plt.rcParams["text.color"] = "white"
```

```
plt.rcParams["axes.labelcolor"] = "white"
plt.rcParams["xtick.color"] = "white"
plt.rcParams["ytick.color"] = "white"
plt.rcParams["axes.facecolor"] = "black"
plt.rcParams["figure.facecolor"] = "black"

# 9. Plot
plt.figure(figsize=(12, 8))
sns.barplot(data=top_20, x='trip_count', y='label', palette=custom_palett
plt.title("Top 20 Most Congested Pickup Hours (Pickup Zone × Hour)", font
plt.xlabel("Number of Trips")
plt.ylabel("Pickup Zone & Hour")
plt.tight_layout()
plt.show()
```

```
/var/folders/p_/tggkw3s56sl80qctb6fwf1nh0000gn/T/ipykernel_81933/320737329
8.py:43: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(data=top_20, x='trip_count', y='label', palette=custom_palet
te)
/var/folders/p_/tggkw3s56sl80qctb6fwf1nh0000gn/T/ipykernel_81933/320737329
8.py:43: UserWarning: Numpy array is not a supported type for `palette`. P
lease convert your palette to a list. This will become an error in v0.14
  sns.barplot(data=top_20, x='trip_count', y='label', palette=custom_palet
te)
```